

Deploy BOWL on a VPS (nginx + gunicorn)

This guide moves the combined hub + three league sites off shared hosting (e.g. PythonAnywhere) onto a small Linux VPS with **nginx** in front and **gunicorn** running `wsgi.application`. The app layout is unchanged: `/`, `/bowl-historical/`, `/bowl-fantasy/`, `/bowl-cap/`.

For CSV import and update cycles after the server exists, keep using [DATA-UPDATE.md](#) and [UPDATE-NESTED-SERVER.md](#) (steps 6–9).

1. VPS sizing (starting point)

Resource	Suggestion
RAM	2 GB minimum (3 league SQLite DBs + site DB + gunicorn workers)
CPU	1–2 vCPU
Disk	20 GB+ (CSV imports, <code>instance/*.db</code> , <code>instance/league_json_cache/</code> , static assets)
OS	Ubuntu 22.04 or 24.04 LTS

2. Server packages

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip nginx git
```

Optional: `certbot python3-certbot-nginx` for Let's Encrypt.

3. App user and checkout

```
sudo adduser --disabled-password bowl
sudo mkdir -p /srv/bowl
sudo chown bowl:bowl /srv/bowl
sudo -u bowl git clone https://github.com/YOUR_ORG/Boys-Of-Winter-League.git
/srv/bowl/app
cd /srv/bowl/app
sudo -u bowl python3 -m venv .venv
sudo -u bowl .venv/bin/pip install -r requirements.txt
sudo -u bowl .venv/bin/pip install gunicorn
```

Copy production env (never commit secrets):

```
sudo -u bowl cp .env.example .env
# edit /srv/bowl/app/.env – see section 5
```

Ensure live databases and CSVs exist under `instance/` and `data/imports/raw/` (rsync or your existing deploy scripts).

4. Environment variables (VPS)

In `/srv/bowl/app/.env` (loaded by the app via `python-dotenv` where configured, or export from systemd):

```
FLASK_DEBUG=0
SECRET_KEY=<long-random-string>
SITE_PUBLIC_BASE_URL=https://www.bowlhockey.com

# Recommended on a dedicated VPS (pre-build homepage JSON after worker start /
import)
LEAGUE_JSON_CACHE_WARM_ON_STARTUP=1

# Optional: paint leaders + standings in HTML from cache (no cold build on home HTML)
HOMEPAGE_SSR_LEADERS_STANDINGS=1

# Site DB – keep MySQL if you already use PA MySQL, or SQLite on VPS:
# SITE_DATABASE_URL=mysql+pymysql://...
```

Leave `LEAGUE_JSON_CACHE_WARM_ON_STARTUP=0` on PythonAnywhere if you stay on PA (many workers × three leagues can spike SQLite I/O on reload).

5. systemd unit (gunicorn)

Create `/etc/systemd/system/bowl-web.service` :

```
[Unit]
Description=BOWL combined WSGI (hub + leagues)
After=network.target

[Service]
User=bowl
Group=bowl
WorkingDirectory=/srv/bowl/app
EnvironmentFile=/srv/bowl/app/.env
Environment=PYTHONPATH=/srv/bowl/app
ExecStart=/srv/bowl/app/.venv/bin/gunicorn \
  --bind 127.0.0.1:8000 \
  --workers 3 \
  --threads 2 \
  --timeout 120 \
  --worker-class gthread \
  --access-logfile - \
  --error-logfile - \
  wsgi:application
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Notes:

- `wsgi:application` is the combined DispatcherMiddleware app (same as PythonAnywhere).
- `--workers 3` : one process can lazy-load each league; 3–4 workers is a reasonable start on 2 GB RAM. Increase only if CPU is idle and RAM allows.
- `--timeout 120` : cold homepage cache builds can be slow; warm cache keeps requests fast.
- First request to a league after a worker fork still pays lazy `create_app` (see `wsgi.py`); warming mitigates JSON endpoints, not that one-time init.

Enable:

```
sudo systemctl daemon-reload
sudo systemctl enable bowl-web
sudo systemctl start bowl-web
sudo systemctl status bowl-web
```

6. nginx site

Create `/etc/nginx/sites-available/bowl`:

```
upstream bowl_app {
    server 127.0.0.1:8000;
    keepalive 8;
}

server {
    listen 80;
    server_name www.bowlhockey.com bowlhockey.com;

    client_max_body_size 20M;

    # Long-cache static assets (combined static middleware + app/static)
    location ~* ^/(bowl-historical|bowl-fantasy|bowl-cap|bowl-formula|bowl-
demolition)?/static/ {
        alias /srv/bowl/app/app/static/;
        try_files $uri @app;
        expires 7d;
        add_header Cache-Control "public, max-age=604800";
    }

    location / {
        proxy_pass http://bowl_app;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
    }

    location @app {
        proxy_pass http://bowl_app;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Enable and test:

```
sudo ln -sf /etc/nginx/sites-available/bowl /etc/nginx/sites-enabled/bowl
sudo nginx -t
sudo systemctl reload nginx
```

TLS: `sudo certbot --nginx -d www.bowlhockey.com -d bowlhockey.com`

Path layout: nginx must forward the **full** path (`/bowl-fantasy/...`) to gunicorn without stripping the league prefix. Do **not** set `SCRIPT_NAME` unless you use an extra URL prefix above the league slugs (see [UPDATE-NESTED-SERVER.md](#)).

Discord interactions and hub routes use `SITE_PUBLIC_BASE_URL` at the domain root (e.g. `https://www.bowlhockey.com/api/discord/interactions`), not under a league prefix.

7. Post-deploy cache warm (recommended)

After `systemctl restart bowl-web` or a DB upload:

```
curl -s -o /dev/null -w "%{http_code} summary\n" \
  "https://www.bowlhockey.com/bowl-fantasy/api/homepage/summary?segment=rs"
curl -s -o /dev/null -w "%{http_code} odds\n" \
  "https://www.bowlhockey.com/bowl-fantasy/api/homepage/postseason-odds"
# Repeat for bowl-historical and bowl-cap
```

Or rely on `LEAGUE_JSON_CACHE_WARM_ON_STARTUP=1` plus import-time warm in `rebuild.py`.

8. uWSGI instead of gunicorn (optional)

If you prefer uWSGI:

```
pip install uwsgi
```

Example ini:

```
[uwsgi]
module = wsgi:application
chdir = /srv/bowl/app
virtualenv = /srv/bowl/app/.venv
master = true
processes = 3
threads = 2
socket = 127.0.0.1:8000
chmod-socket = 660
vacuum = true
die-on-term = true
harakiri = 120
env = PYTHONPATH=/srv/bowl/app
```

Point nginx `proxy_pass` at the socket or HTTP listener. Gunicorn is usually simpler for Flask; both work with the same `wsgi:application`.

9. Discord bot and cron on the VPS

Run the league Discord bot as a **separate** systemd service (same repo, same `.env`), pointing `DISCORD_BOT_LEAGUE_BASE_URLS` at your public URLs.

Keep using `scripts/run_site_update.py` from your dev machine or a CI runner; on the VPS you only need `git pull` / `rsync` DBs, imports, and `systemctl restart bowl-web`.

10. Migration checklist from PythonAnywhere (summary)

Detailed cutover steps: **section 11**.

1. Copy live data and secrets from PA → VPS (§11.2–11.3).
 2. Build VPS stack (sections 2–6), test in parallel (§11.4).
 3. Cut DNS (§11.5), move Discord bot (§11.6), repoint deploy (§11.7).
 4. Decommission PA when stable (§11.8).
-

11. PythonAnywhere → VPS cutover (runbook)

This section turns the current **bowlhockey.com on PythonAnywhere** setup into the nginx + gunicorn VPS layout without changing URLs (`/`, `/bowl-fantasy/`, etc.) or stat/import logic.

11.1 What lives on PythonAnywhere today

Typical production layout (defaults in `scripts/STEP2_pythonanywhere.py` and [scripts/README.md](#)):

Item	PythonAnywhere path
Project root	<code>/home/BoiledEgg1974/boys-of-winter-hockey-website</code>
Virtualenv	<code>/home/BoiledEgg1974/venv</code>
WSGI reload file	<code>/var/www/www_bowlhockey_com_wsgi.py</code>
SSH	<code>ssh.pythonanywhere.com</code> , user <code>BoiledEgg1974</code>

Item	PythonAnywhere path
League SQLite	<code>instance/bowl-historical.db</code> , <code>bowl-fantasy.db</code> , <code>bowl-cap.db</code> (+ racing if used)
Site DB	<code>instance/site_membership.db</code> or <code>SITE_DATABASE_URL</code> → PA MySQL
JSON cache	<code>instance/league_json_cache/</code>
CSV exports	<code>data/imports/raw/bowl_historical/</code> , <code>bowl_fantasy/</code> , <code>bowl_cap/</code>
Secrets / env	<code>boys-of-winter-hockey-website/.env</code> (Web tab vars should match this file)
Discord bot	Always-on task: <code>python -m scripts.league_discord_bot</code> (see DISCORD BOT SETUP.md)

Nightly updates from your PC use `python scripts/BOWL-Site-Update.py` , which runs local imports then `STEP2_pythonanywhere.py deploy-db` : SSH to PA, **capture live OVR / trade log / editorial state**, merge into local DBs, upload SQLite + static, run `notify_discord_after_db_deploy.py` , touch WSGI to reload.

The VPS target keeps the **same repo** and `wsgi:application` ; only the process manager and deploy reload step change (systemd instead of `touch ...wsgi.py`).

11.2 Phase A — Build the VPS from a PA snapshot (one time)

Do this **before** DNS cutover so the VPS is a faithful copy of live.

On your PC (Git Bash or WSL; install OpenSSH). Set key-based SSH to both hosts (`PA_SSH_KEY` as you use for STEP2).

```
PA_USER=BoiledEgg1974
PA_HOST=ssh.pythonanywhere.com
PA_ROOT=/home/BoiledEgg1974/boys-of-winter-hockey-website
VPS_USER=bowl
VPS_HOST=YOUR.VPS.IP
VPS_ROOT=/srv/bowl/app
```

1. **Code** — on the VPS (section 3): `git clone` the same branch you deploy on PA, `pip install -r requirements.txt gunicorn` .
2. **Databases + cache** — pull from PA (does not delete PA data):

```
rsync -avz -e "ssh -i $PA_SSH_KEY" \
"$PA_USER@$PA_HOST:$PA_ROOT/instance/" \
"$VPS_USER@$VPS_HOST:$VPS_ROOT/instance/"

rsync -avz -e "ssh -i $PA_SSH_KEY" \
"$PA_USER@$PA_HOST:$PA_ROOT/data/imports/raw/" \
"$VPS_USER@$VPS_HOST:$VPS_ROOT/data/imports/raw/"
```

Include `instance/league_json_cache/` in the first rsync if present (faster first homepage load).

1. **Static assets** (logos, headshots, CSS you did not git-track):

```
rsync -avz -e "ssh -i $PA_SSH_KEY" \
"$PA_USER@$PA_HOST:$PA_ROOT/app/static/" \
"$VPS_USER@$VPS_HOST:$VPS_ROOT/app/static/"
```

1. **Environment file** — copy PA `.env` to the VPS and edit only what must change:

```
scp -i "$PA_SSH_KEY" \
"$PA_USER@$PA_HOST:$PA_ROOT/.env" \
"$VPS_USER@$VPS_HOST:$VPS_ROOT/.env"
```

On the VPS `.env`, set at minimum:

- `FLASK_DEBUG=0`
- `SITE_PUBLIC_BASE_URL=https://www.bowlhockey.com` (unchanged until you use a staging hostname)
- `LEAGUE_JSON_CACHE_WARM_ON_STARTUP=1`
- Optional: `HOMEPAGE_SSR_LEADERS_STANDINGS=1`

Keep `SECRET_KEY`, `DISCORD_EVENTS_SHARED_SECRET`, mail settings, and `SITE_DATABASE_URL` the same as PA if you are not moving the site DB yet.

1. Finish sections **5–6** (systemd + nginx). Use a **staging** `server_name` or test via `/etc/hosts` (§11.4) before pointing production DNS.

11.3 Site database (MySQL vs SQLite)

Situation	Action
PA uses MySQL (<code>SITE_DATABASE_URL</code> in <code>.env</code>)	Easiest cutover: keep the same MySQL URL from the VPS (PA MySQL allows remote if whitelisted; or migrate DB to VPS MySQL).

Situation	Action
PA uses SQLite instance/site_membership.db	Rsync copied it in §11.2; no extra step.
Move MySQL onto the VPS	One-time: <code>.env.example</code> → <code>migrate_site_sqlite_to_mysql.py</code> / your host's MySQL docs; then point <code>SITE_DATABASE_URL</code> at the new server and re-test GM login, news, Discord queue.

League stats stay in **per-league SQLite** under `instance/`; only the **site** DB holds GM accounts, news queue, Discord integration, etc.

11.4 Phase B — Parallel test (no DNS change yet)

1. On your PC, map the VPS IP to a fake name, e.g. `104.x.x.x staging.bowlhockey.com` in `C:\Windows\System32\drivers\etc\hosts`.
2. Temporarily set nginx `server_name staging.bowlhockey.com`; and obtain TLS, **or** test over HTTP on the IP with `curl -H "Host: www.bowlhockey.com" http://VPS_IP/bowl-fantasy/`.
3. Smoke-test the same list as [UPDATE-NESTED-SERVER.md](#) §9: hub, each league home, standings, player search, GM login, admin Discord test event.
4. Compare one heavy page (e.g. team page, homepage after cache warm) against PA.

Fix nginx paths, `.env`, and worker count before cutover.

11.5 Phase C — Production cutover (DNS)

Namecheap step-by-step: §13.

1. **Lower TTL** on `bowlhockey.com` / `www` a day ahead (e.g. 300s) if your DNS provider allows.
2. **Stop sending users to PA:** update **A/AAAA** for `www.bowlhockey.com` (and apex if used) to the **VPS**.
3. Run **certbot** on the VPS for the real `server_name` (section 6).
4. **Restart web** and run cache warm (section 7).
5. Smoke-test from a network **without** `/etc/hosts` overrides.
6. Leave the PA web app **enabled but unused** for 24–48h (rollback, §11.9).

Discord interactions URL stays `https://www.bowlhockey.com/api/discord/interactions` — no Developer Portal change if the domain is unchanged.

11.6 Discord delivery bot (Always-on → systemd)

1. **Before cutover:** note PA Always-on command (usually):

```
cd /home/BoiledEgg1974/boys-of-winter-hockey-website &&  
/home/BoiledEgg1974/venv/bin/python -m scripts.league_discord_bot
```

1. On the VPS, create `/etc/systemd/system/bowl-discord-bot.service` :

```
[Unit]  
Description=BOWL league Discord delivery bot  
After=network.target bowl-web.service  
  
[Service]  
User=bowl  
Group=bowl  
WorkingDirectory=/srv/bowl/app  
EnvironmentFile=/srv/bowl/app/.env  
Environment=PYTHONPATH=/srv/bowl/app  
ExecStart=/srv/bowl/app/.venv/bin/python -m scripts.league_discord_bot  
Restart=on-failure  
RestartSec=10  
  
[Install]  
WantedBy=multi-user.target
```

1. `sudo systemctl enable --now bowl-discord-bot` .
2. Admin → Discord integration → **Queue test event** on each league.
3. **After the bot is healthy on the VPS**, disable the PA Always-on task (only one poller should run).

Ensure `.env` on the VPS includes `DISCORD_BOT_TOKEN` , `DISCORD_EVENTS_SHARED_SECRET` , and `SITE_PUBLIC_BASE_URL` (same as web).

11.7 Phase D — Repoint nightly deploy (after DNS points to VPS)

Your local pipeline can **keep using STEP2**; point SSH env vars at the **VPS** instead of PythonAnywhere. The remote promote + Discord notify scripts are the same; only the reload step differs.

On your Windows PC (PowerShell — persist in your profile or a `deploy-vps.env` you source):

```
$env:PA_HOST = "YOUR.VPS.IP"           # or vps.bowlhockey.com  
$env:PA_USER = "bowl"  
$env:PA_REMOTE_PATH = "/srv/bowl/app"  
$env:PA_REMOTE_VENV_BIN = "/srv/bowl/app/.venv/bin"  
$env:PA_SSH_KEY = "$HOME\.ssh\id_ed25519_vps" # your VPS key
```

Then run the usual update, but **skip WSGI touch** and restart gunicorn over SSH:

```
python scripts/BOWL-Site-Update.py --deploy-db-only
# STEP2 post-upload script touches PA WSGI; on VPS use --skip-reload:
python scripts/STEP2_pythonanywhere.py deploy-db --skip-reload
ssh -i $env:PA_SSH_KEY bowl@YOUR.VPS.IP "sudo systemctl restart bowl-web"
```

Recommended habit after **BOWL-Site-Update.py** (full nightly): wrap STEP2 with `--skip-reload` and restart, e.g. adjust your wrapper or run:

```
python scripts/BOWL-Site-Update.py --no-deploy
python scripts/STEP2_pythonanywhere.py deploy-db --skip-reload
ssh bowl@YOUR.VPS.IP "sudo systemctl restart bowl-web"
```

(`--no-deploy` skips the built-in PA deploy at the end of BOWL-Site-Update; you run STEP2 yourself with VPS env + `--skip-reload` .)

sudo: allow passwordless restart for `bowl` :

```
/etc/sudoers.d/bowl-web : bowl ALL=(ALL) NOPASSWD: /bin/systemctl restart bowl-web
```

What still runs on the server during `deploy-db` : capture live OVR/trade/editorial → merge locally → upload SQLite → `notify_discord_after_db_deploy.py` . That logic is unchanged; it now runs against the VPS tree under `/srv/bowl/app` .

Code-only changes (no DB upload): `git pull` on the VPS + `systemctl restart bowl-web` , or extend your workflow with `STEP2 sync` targeting the VPS (same `PA_*` overrides, no import).

11.8 Decommission PythonAnywhere

When the VPS has been stable through at least one full **BOWL-Site-Update** cycle:

1. Disable PA **Web** app and **Always-on** task.
2. Keep a final backup: `python scripts/backup_all_live_data.py` on PA or `rsync instance/` once more.
3. Optionally downgrade/cancel PA hosting.

Do **not** delete PA until league SQLite and site DB backups are verified on the VPS.

11.9 Rollback

If something fails right after DNS change:

1. Point DNS A/AAAA back to PythonAnywhere.
2. Re-enable PA Web + Always-on bot; disable VPS bot/web if they might conflict.
3. If you uploaded DBs to the VPS during a bad deploy, restore PA from your last good `instance/` backup — PA was unchanged during parallel testing if you did not run

deploy-db against it after cutover.

11.10 Cutover timeline (example)

Day	Action
D-3	Provision VPS, sections 2-6, rsync from PA (§11.2)
D-2	Parallel testing (§11.4), fix nginx/env
D-1	Lower DNS TTL; dry-run <code>deploy-db --skip-reload</code> to VPS
D0	DNS to VPS (§11.5); start VPS Discord bot; disable PA bot
D+1	Full <code>BOWL-Site-Update</code> with VPS STEP2 (§11.7)
D+7	Decommission PA (§11.8)

12. DigitalOcean Droplet created — first-hour checklist

Use this right after you create a Droplet (e.g. **Basic 4 GB / 2 vCPU, Ubuntu 24.04, nyc1**). Replace `YOUR.DROPLET.IP` and SSH user (`root` on first login, then `bow1` once created).

12.1 On the Droplet (SSH as root)

```
# 1 - Base packages
apt update && apt install -y python3 python3-venv python3-pip nginx git ufw certbot
python3-certbot-nginx

# 2 - App user + directory
adduser --disabled-password --gecos "" bow1
mkdir -p /srv/bow1 && chown bow1:bow1 /srv/bow1

# 3 - Clone app (use your real repo URL)
sudo -u bow1 git clone https://github.com/YOUR_ORG/Boys-Of-Winter-League.git
/srv/bow1/app
cd /srv/bow1/app && sudo -u bow1 python3 -m venv .venv
sudo -u bow1 .venv/bin/pip install -r requirements.txt gunicorn

# 4 - Firewall (SSH first, then web)
ufw allow OpenSSH && ufw allow 'Nginx Full' && ufw --force enable

# 5 - Copy .env from PA later (§11.2); for now:
sudo -u bow1 cp .env.example .env
# edit: FLASK_DEBUG=0, SECRET_KEY, SITE_PUBLIC_BASE_URL,
LEAGUE_JSON_CACHE_WARM_ON_STARTUP=1
```

Install **systemd** + **nginx** from **sections 5–6** (paths `/srv/bowl/app`). Do **not** point production DNS yet.

12.2 One-time data copy from PythonAnywhere (your PC)

```
# 6 – League DBs, site DB, JSON cache (Git Bash / WSL; set PA_SSH_KEY)
rsync -avz -e "ssh -i $PA_SSH_KEY"
BoiledEgg1974@ssh.pythonanywhere.com:/home/BoiledEgg1974/boys-of-winter-hockey-
website/instance/ bowl@YOUR.DROPLET.IP:/srv/bowl/app/instance/

# 7 – CSV + static
rsync -avz -e "ssh -i $PA_SSH_KEY"
BoiledEgg1974@ssh.pythonanywhere.com:/home/BoiledEgg1974/boys-of-winter-hockey-
website/data/imports/raw/ bowl@YOUR.DROPLET.IP:/srv/bowl/app/data/imports/raw/
rsync -avz -e "ssh -i $PA_SSH_KEY"
BoiledEgg1974@ssh.pythonanywhere.com:/home/BoiledEgg1974/boys-of-winter-hockey-
website/app/static/ bowl@YOUR.DROPLET.IP:/srv/bowl/app/app/static/

# 8 – Production .env (secrets)
scp -i "$PA_SSH_KEY" BoiledEgg1974@ssh.pythonanywhere.com:/home/BoiledEgg1974/boys-
of-winter-hockey-website/.env bowl@YOUR.DROPLET.IP:/srv/bowl/app/.env
```

On the VPS, edit `.env`: set `LEAGUE_JSON_CACHE_WARM_ON_STARTUP=1` (and optional `HOMEPAGE_SSR_LEADERS_STANDINGS=1`).

12.3 Start web + smoke test (before Namecheap cutover)

```
# 9 – On VPS
sudo systemctl enable --now bowl-web
sudo nginx -t && sudo systemctl reload nginx

# 10 – From your PC (replace IP; simulates Host header until DNS moves)
curl -sI -H "Host: www.bowlhockey.com" http://YOUR.DROPLET.IP/bowl-fantasy/ | head -5
```

Test in a browser via `/etc/hosts` (§11.4): map `YOUR.DROPLET.IP` → `www.bowlhockey.com`. When happy, follow **§13** for Namecheap DNS, then **§11.5–11.7** (cutover, Discord bot, deploy-db to VPS).

13. Namecheap — domain, DNS, and cutover from PythonAnywhere

You usually **keep the domain registered at Namecheap** and only **change DNS** so `bowlhockey.com` / `www` point to the DigitalOcean Droplet. You do **not** need to transfer the

domain to DigitalOcean unless you want Namecheap purely as registrar and DO to host DNS (optional, §13.3).

13.1 What stays where

Piece	Where it lives
Domain registration	Namecheap (renewal, WHOIS, domain lock)
Website + app	DigitalOcean Droplet (nginx + gunicorn)
Old host (during migration)	PythonAnywhere until you disable it after cutover
Discord Developer Portal	No change if the public URL stays <code>https://www.bowlhockey.com/...</code>

13.2 Find your Droplet IP

DigitalOcean → **Droplets** → your server → copy **Public IPv4** (e.g. `157.x.x.x`). You will create DNS records pointing to this address.

13.3 Two ways to manage DNS (pick one)

Option A — Keep DNS at Namecheap (simplest for most)

1. Namecheap → **Domain List** → **Manage** next to `bowlhockey.com`.
2. **Advanced DNS** tab.
3. Remove or edit old records that pointed to **PythonAnywhere** (often an **A** record for `@` and/or **CNAME** for `www` to PA or parking).
4. Add/update:

Type	Host	Value	TTL
A	<code>@</code>	<code>YOUR.DROPLET.IP</code>	300 (before cutover) → 3600 after stable
A	<code>www</code>	<code>YOUR.DROPLET.IP</code>	same

Some setups use **CNAME** `www` → `@`; Namecheap supports that if `@` is an A record to the Droplet IP.

Option B — Use DigitalOcean nameservers

1. DO → **Networking** → **Domains** → **Add Domain** → `bowlhockey.com`.
2. Create the same **A** records (`@` and `www` → Droplet IP) in DO.
3. Namecheap → **Domain** → **Nameservers** → **Custom DNS** → set to DO's nameservers (e.g. `ns1.digitalocean.com`, `ns2.digitalocean.com`, `ns3.digitalocean.com`).
4. Propagation can take up to 24–48 hours; until then, either nameserver set is "live."

For a single site, **Option A** is enough.

13.4 Before you change DNS (reduce downtime)

1. **Lower TTL** on existing `@` / `www` records to **300 seconds** (5 min) at Namecheap **24–48 hours before** cutover.
2. Finish §12 and §11.4 parallel testing (hosts file or staging).
3. On the VPS, ensure nginx `server_name` includes `www.bowlhockey.com` and `bowlhockey.com` (section 6).
4. Obtain TLS **after** DNS points to the Droplet (or use hosts-file testing only on HTTP until cutover):

```
sudo certbot --nginx -d www.bowlhockey.com -d bowlhockey.com
```

Certbot needs the world to resolve the name to your Droplet for the HTTP challenge (unless you use DNS challenge — not covered here).

13.5 Cutover day (Namecheap + DO)

1. Update **A** records (`@` and `www`) to the **Droplet IP** (§13.3).
2. Wait for TTL (often 5–30 minutes if you lowered TTL).
3. Verify:

```
nslookup www.bowlhockey.com
curl -sI https://www.bowlhockey.com/bowl-fantasy/
```

1. Run **cache warm** (section 7).
2. Start **Discord bot** on VPS (§11.6); disable PA Always-on.
3. `SITE_PUBLIC_BASE_URL` in `.env` must stay `https://www.bowlhockey.com` (no trailing slash).

13.6 Do not break email (if you use Namecheap mail)

If `@` uses Namecheap **Email Forwarding** or **Private Email**, **do not delete MX records** when editing Advanced DNS. Typical pattern:

- Leave **MX** (and any **TXT** for SPF/DKIM) untouched.
- Only change **A** / **CNAME** for the **website** hostnames (`@`, `www`).

If you are unsure, screenshot Advanced DNS before editing.

13.7 Apex vs www (redirect)

Browsers and Discord links may use either `bowlhockey.com` or `www.bowlhockey.com`. Best practice:

- **A** records for **both** `@` and `www` to the Droplet, **or** `@` A + `www` CNAME to `@`.
- In nginx, one `server` block with both names (section 6) and certbot for both `-d` flags.

Your Flask app expects the same paths on both; hub is at `/`.

13.8 PythonAnywhere after DNS moves

Traffic goes to DO; PA is idle unless someone uses the old IP directly.

1. Keep PA **read-only** for a few days (rollback §11.9).
2. Repoint `deploy-db` to the VPS (§11.7).
3. Disable PA web + Always-on when stable (§11.8).

You do **not** need to “transfer the domain” away from Namecheap for this migration.

13.9 Optional: move only DNS hosting to DigitalOcean

Registrar stays Namecheap; nameservers switch to DO (§13.3 Option B). Useful if you later add DO **Load Balancers** or multiple Droplets. Not required for one Droplet.

13.10 Troubleshooting

Symptom	Check
Site still opens on PA	DNS cache / old TTL; <code>nslookup</code> from phone off Wi-Fi; flush local DNS
Certificate fails	A record not pointing to Droplet yet; wait propagation
<code>www</code> works, apex does not	Missing A for <code>@</code> or wrong IP
Mixed content / wrong league paths	nginx must proxy full path; see section 6
Email stopped	MX records changed by mistake — restore from screenshot

Quick reference

Component	Value
WSGI entry	<code>wsgi:application</code>
Working directory	project root (contains <code>wsgi.py</code>)
JSON cache dir	<code>instance/league_json_cache/</code>

Component	Value
League DBs	instance/bowl-historical.db , bowl-fantasy.db , bowl-cap.db
Site DB	instance/site_membership.db or SITE_DATABASE_URL